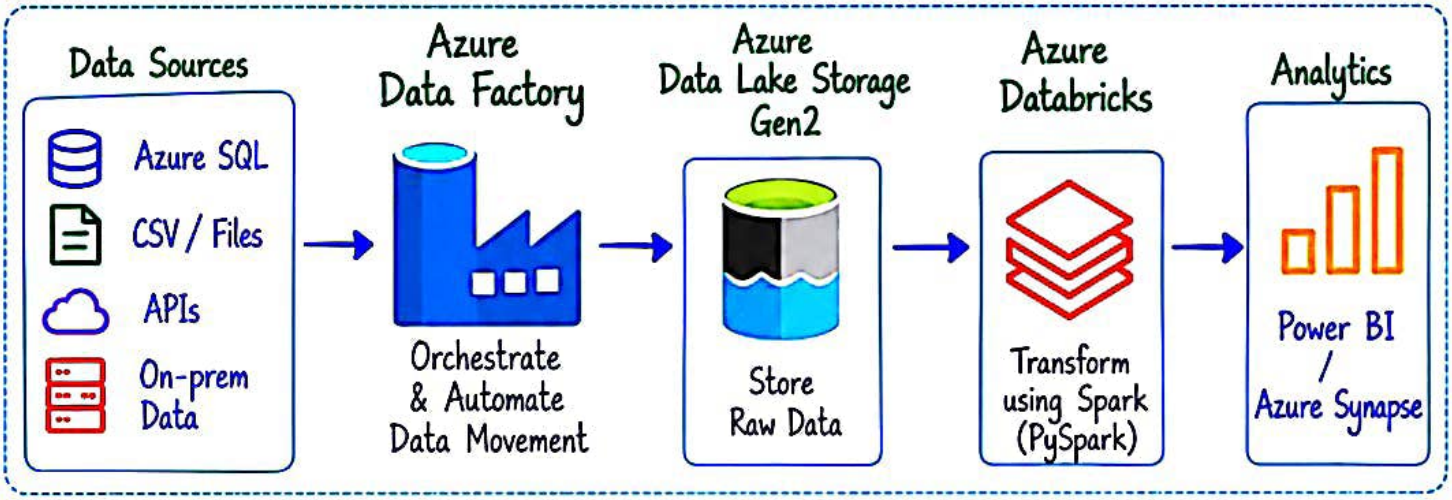
NEERAJ SINGH
DEVOPS

30 Days Azure Data Engineering Challenge

Day 13 of 30

Azure Data Factory

Pipelines • Activities • Triggers • Integration Runtime



Ingest	Orchestrate	Store	Transform	Analyze
• Multiple sources • Batch / Incremental	• Pipelines • Activities • Triggers	• ADLS Gen2 • Raw / Bronze Layer	• Azure Databricks • Data Processing • Silver / Gold Layer	• Power BI • Azure Synapse • Reports & Dashboards

From Data Sources to Automated Pipelines

Azure Data Factory Fundamentals

A fully managed cloud service for data integration and orchestration

1 What is Azure Data Factory?

- Azure Data Factory (ADF) is a fully managed cloud data integration and orchestration service.
- Used to create, schedule and monitor data pipelines to move and transform data at scale.
- Supports 100+ connectors to on-premises, cloud and SaaS data sources.
- Helps in building ETL and ELT workflows without managing infrastructure.

2 Key Features

- **Data Movement** – Move data between 100+ sources using built-in connectors.
- **Scheduling & Orchestration** – Automate pipelines using triggers.
- **Data Transformation** – Integrate with Databricks, Synapse, Mapping Data Flows.
- **Hybrid Connectivity** – Connect to on-premises data sources using Self-hosted Integration Runtime.
- **Monitoring & Alerting** – Track pipeline runs, set alerts and handle failures.
- **Scalable** – Serverless and fully managed by Azure.

3 ETL vs ELT

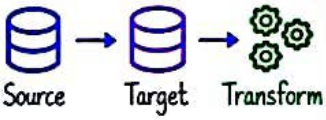
ETL (Extract, Transform, Load)

- Transform data before loading into target.
- Used when target system has limited processing power.
- Common for data warehouses.



ELT (Extract, Load, Transform)

- Load data first, then transform in target system.
- Used when target has high processing power (e.g., Databricks, Synapse).
- Common in modern cloud data platforms.



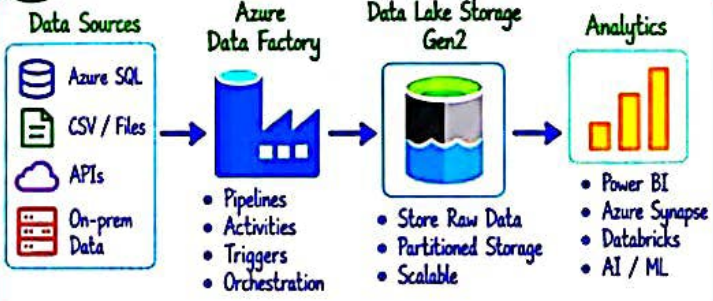
4 Core Components of ADF

- **Pipelines** – Logical grouping of activities.
- **Activities** – Individual processing steps like Copy, Lookup, ForEach, etc.
- **Linked Services** – Connection information for data stores and compute services.
- **Datasets** – Reference to data within a connected data store.
- **Integration Runtime** – Compute infrastructure for data movement and activity execution.
- **Triggers** – Start pipelines automatically on schedule or events.

5 Common Use Cases

- **Database Ingestion**
  Load data from Azure SQL, MySQL, Oracle, etc.
- **File Processing**
  Ingest CSV, Parquet, JSON, Excel files.
- **API Integration**
  Load data from REST APIs and SaaS applications.
- **Hybrid Data Integration**
  Connect to on-premises databases.
- **Incremental Data Loading**
  Use watermark or change tracking.

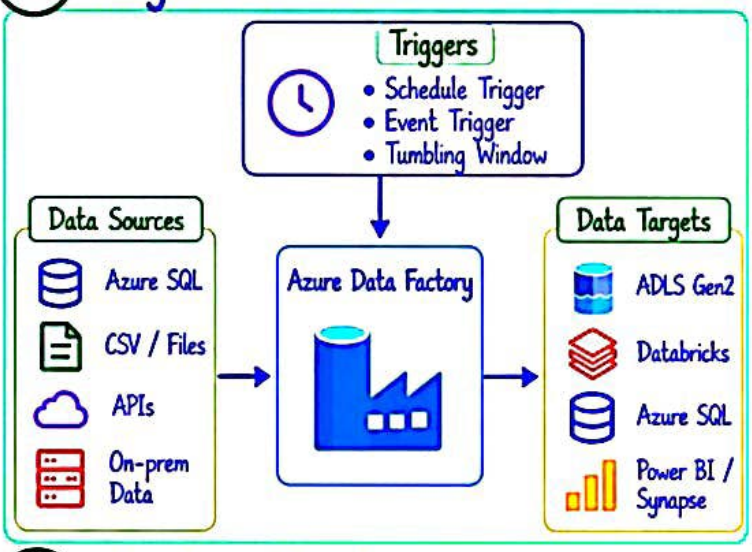
6 End-to-End Data Pipeline with ADF



Azure Data Factory Architecture

Main components and how they work together

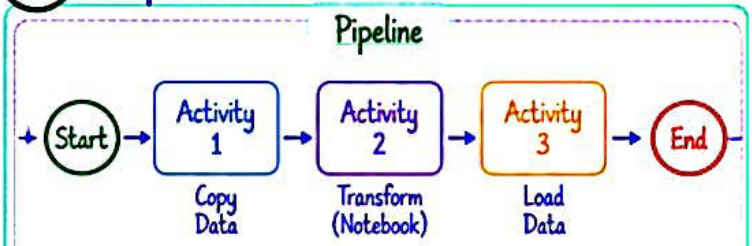
1 High Level Architecture



2 Components in ADF

Pipeline	• Logical grouping of activities • Defines the workflow
Activity	• Individual processing step • Examples: Copy, Lookup, ForEach, etc.
Linked Service	• Connection information for data stores and compute services
Dataset	• Reference to data within a connected store • Defines structure, format and location
Integration Runtime	• Compute infrastructure for data movement and activity execution
Trigger	• Start pipeline automatically or on demand • Types: Schedule, Event, Tumbling Window

3 Pipeline Structure



- A pipeline can have multiple activities running in sequence or parallel.
- Activities can be connected using dependencies (Success, Failure, Completion, etc.).

4 Datasets and Linked Services

Linked Services

- Define connection to external data source or compute.
- Store connection details like URL, authentication, credentials.

Examples:

Azure SQL · ADLS Gen2 · Databricks · REST API

Datasets

- Reference to data within a linked service.
- Define data format, schema, file path or table name.

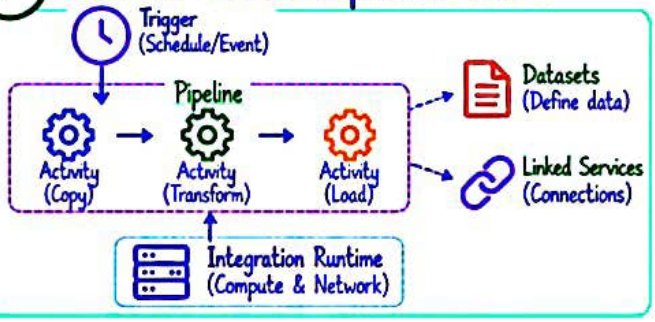
Examples:

DelimtedText (CSV) · JSON · Parquet · Azure SQL Table

5 Triggers in ADF

Schedule Trigger	Tumbling Window Trigger	Event Trigger
• Run pipeline on a schedule • e.g. every day every hour	• Process fixed time windows • Handles late data • Used for incremental loads	• Start pipeline on events (e.g. file arrival in ADLS Gen2) • Near real-time processing

6 End-to-End Component Flow



Integration Runtime in Azure Data Factory

Compute infrastructure for data movement and activity execution

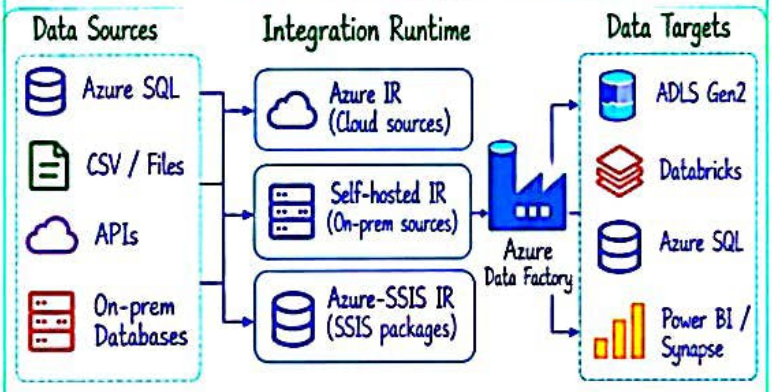
1 What is Integration Runtime (IR)?

- Integration Runtime (IR) is the compute infrastructure used by Azure Data Factory to provide data movement and activity execution capabilities.
- It acts as a bridge between ADF and your data sources, whether in Azure, on-premises or other clouds.
- ADF can have multiple IRs in the same factory based on different use cases.

2 Types of Integration Runtime

Azure Integration Runtime (Azure IR)	• Managed by Microsoft • Used for cloud data sources • No installation required
Self-hosted Integration Runtime (SHIR)	• Customer managed • Used for on-premises or private network data sources • Requires installation on a VM
Azure-SSIS Integration Runtime (SSIS IR)	• Run existing SSIS packages in Azure • Used for migration and lift-and-shift scenarios

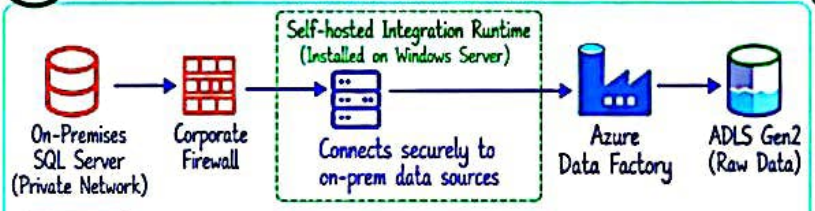
3 Architecture with Integration Runtime



4 Comparison of Integration Runtimes

Feature	Azure IR	Self-hosted IR	Azure-SSIS IR
Managed by	Microsoft	Customer	Microsoft
Deployment	No setup	Install on VM	Managed in Azure
Use Case	Cloud data sources	On-prem / private sources	Run SSIS packages
Connectivity	Public endpoints in Azure	On-prem, private network, VNet	Connect to wide range (SSIS)
Scalability	Auto-scale	Manual (scale out)	Configurable (2-32 cores)
Cost	Pay per use	No additional cost (VM cost only)	Charged per hour
Examples	Azure SQL, ADLS, Blob, REST APIs	On-prem SQL, File servers, SFTP	SSIS packages (ETL workloads)

5 Real-World Scenario: On-Prem to ADLS Gen2



Steps:

1. Install Self-hosted IR on a machine with access to on-premises network.
2. Register the IR with Azure Data Factory.
3. Create linked service for on-prem SQL Server using the Self-hosted IR.
4. Configure a Copy activity to load data to ADLS Gen2.
5. Schedule the pipeline and monitor the runs.

6 Best Practices & Considerations

- ✔ Use Azure IR for cloud-to-cloud data movement.
- ✔ Use Self-hosted IR for on-premises or private network sources.
- ✔ Keep Self-hosted IR running (use high availability for production).
- ✔ Use multiple IR nodes for better performance and load balancing.
- ✔ Ensure network connectivity and firewall rules are configured.
- ✔ Use Managed Identity or Key Vault for secure authentication.
- ✔ Monitor IR performance, CPU, memory and queue length.
- ✔ Keep IR version up to date.

Important Notes:

- Self-hosted IR should be installed on a machine that can access the data source (not on your local laptop).
- Azure IR cannot directly access on-premises data sources.
- You can have multiple IRs in a single ADF instance.

ADF Activities and Data Flows

Common activities used in Azure Data Factory to build data pipelines

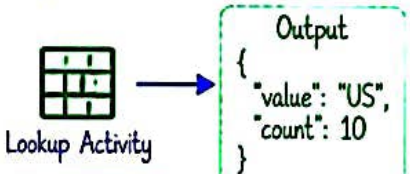
1 Copy Activity

- Used to move data between supported source and sink.
- Supports 100+ connectors (Azure SQL, ADLS, S3, APIs, etc).
- Handles large scale data movement with high performance.
- Supports full and incremental loads.



2 Lookup Activity

- Used to read a small amount of data (like configuration or metadata).
- Returns a single value or an array.
- Commonly used for dynamic parameters and control flow.

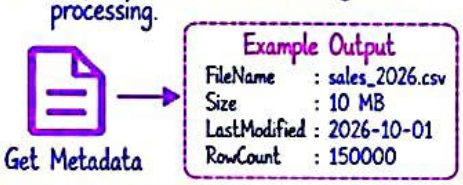


Use Cases:

- Read watermark value.
- Get file list from a table.
- Fetch configuration values.

3 Get Metadata Activity

- Used to retrieve metadata about files, folders or datasets.
- Can get file name, size, last modified time, row count, etc.
- Helps in control flow and dynamic processing.

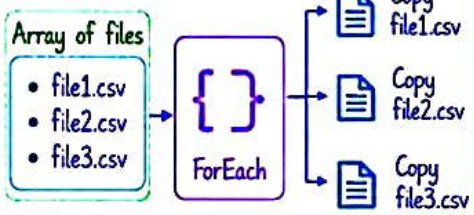


Common Properties:

- childItems - list files in a folder
- exists - check if file exists
- size - return file size
- lastModified - last modified timestamp

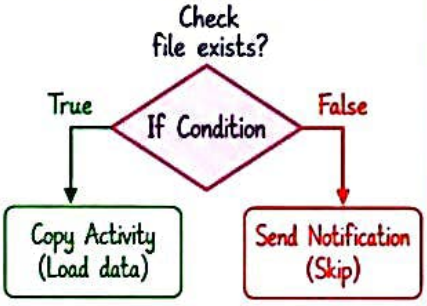
4 ForEach Activity

- Used to iterate over a collection of items (array or list).
- Process multiple files, folders or configuration values.
- Can run activities sequentially or in parallel.



5 If Condition Activity

- Used to implement conditional logic in pipelines.
- Execute different activities based on a condition.
- Supports expressions, parameters and activity output values.



6 Mapping Data Flow

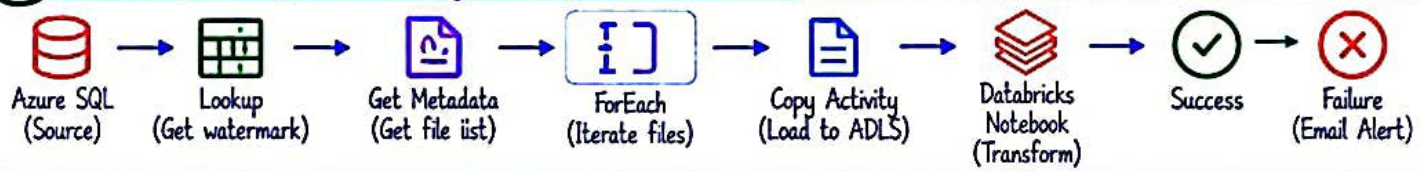
- Code-free, visual data transformation using Spark.
- Used for complex transformations (cleanse, join, aggregate, filter, etc).
- Automatically scales with large datasets.
- Ideal for ETL/ELT scenarios.



Transformations:

- Filter
- Join
- Aggregate
- Derived Column
- Lookup
- Union
- Sort
- Conditional Split
- Sink

7 Example Pipeline using Multiple Activities



ADF Triggers and Parameterization

Automate, Schedule and Build Flexible Pipelines

1 Types of Triggers

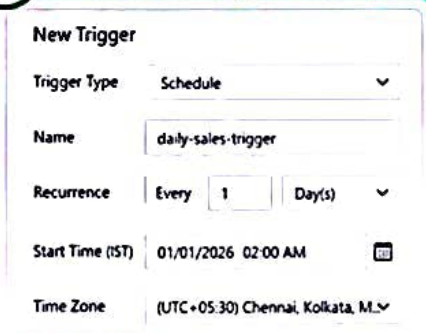
Schedule Trigger
- Run pipeline on a schedule
- e.g. daily, hourly, weekly

Tumbling Window Trigger
- Process fixed time windows
- Handles late data, backfill

Event Trigger
- Start pipeline on an event
- e.g. file arrival in ADLS Gen2

Manual Trigger
- Run pipeline on demand
- Used for testing / ad-hoc runs

2 Schedule Trigger Example



Use Cases:
- Daily incremental data load
- Run reports generation
- Automate ETL pipelines

3 Tumbling Window Trigger



- Runs pipeline for fixed time intervals
- Maintains state and processes each window
- Supports backfill for missed windows
- Ideal for incremental and batch processing

Common Settings:
- Start Time
- Frequency (Hour / Day)
- Window Size
- Advanced → Retry, Timeout

4 Event Trigger (Storage Events)



Supported Events:
- Blob created (new file)
- Blob deleted
- Blob renamed
- Works with ADLS Gen2 and Blob Storage

5 Pipeline Parameters

- Used to pass values to pipelines at runtime.
- Make pipelines reusable.
- Can be used in activities, datasets and dynamic content.

Example Parameters:
- sourcePath (String)
- fileName (String)
- loadDate (String)
- environment (String)
- batchSize (Int)

6 Variables vs Parameters

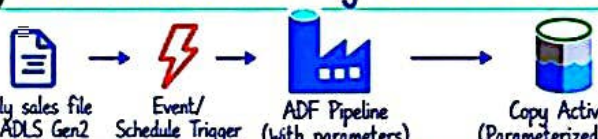
Pipeline Parameters	Variables
• Input values to pipeline	• Used inside pipeline
• Set while trigger or during execution	• Can be changed during execution
• Immutable (cannot be changed)	• Mutable
• Data types: String, Int, Bool, Array, etc.	• Data types: String, Int, Bool, Array, etc.

7 Dynamic Content and Expressions

- Use dynamic content to build values using parameters, variables and system functions.
- Commonly used in datasets, activities and linked services.

Example Expressions:
- @pipeline().parameters.fileName
- @variables('folderPath')
- @concat('sales//, formatDateTime(utcNow(), 'yyyy/MM/dd'))
- @activity('Lookup1').output.firstRow.value

8 Real-World Scenario: Daily Sales File Processing



Dynamic File Path Example:

```
@concat(
    'sales/',
    formatDateTime(utcNow(), 'yyyy/MM/dd'),
    '/',
    pipeline().parameters.fileName
)
```

Benefits:
- Automated file processing
- Reusable pipelines
- Date-based folder structure
- Supports incremental loads
- Easier maintenance

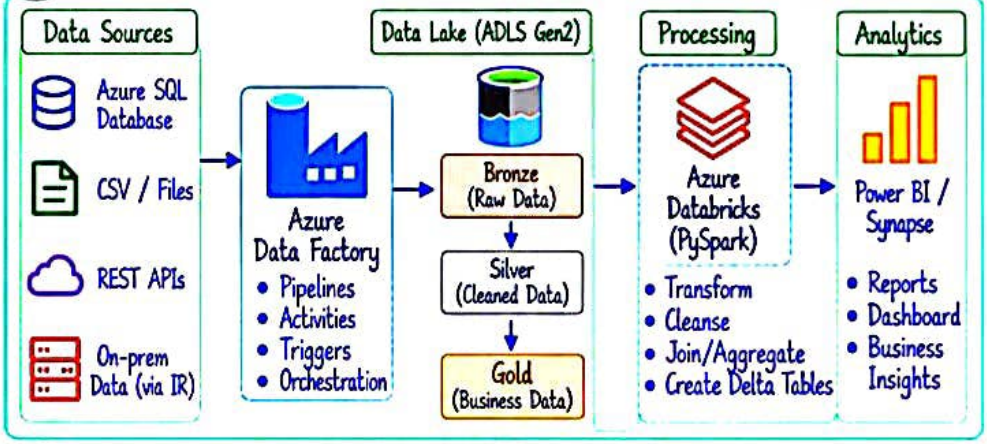
Real-World ADF Pipeline for Data Engineering

From Source to Analytics using Azure Data Factory, Databricks and Delta Lake

1 Business Scenario

- Ingest daily sales data from Azure SQL database.
- Store raw data in ADLS Gen2 (Bronze layer).
- Transform data using Azure Databricks (PySpark).
- Store cleaned data in Delta Lake (Silver and Gold layers).
- Provide data for analytics using Power BI.

2 End-to-End Pipeline Architecture



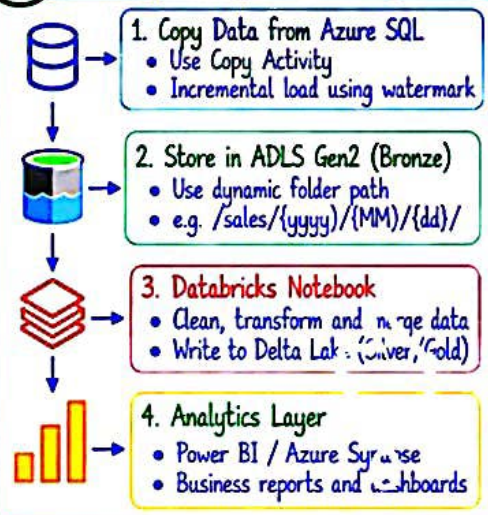
3 Pipeline Flow

1. Extract data from Azure SQL using Copy Activity.
2. Store raw data in ADLS Gen2 (Bronze) with date-based folder.
3. Trigger Databricks Notebook for data transformation.
4. Write transformed data to Delta Lake (Silver/Gold).
5. Data available for analytics in Power BI / Synapse.

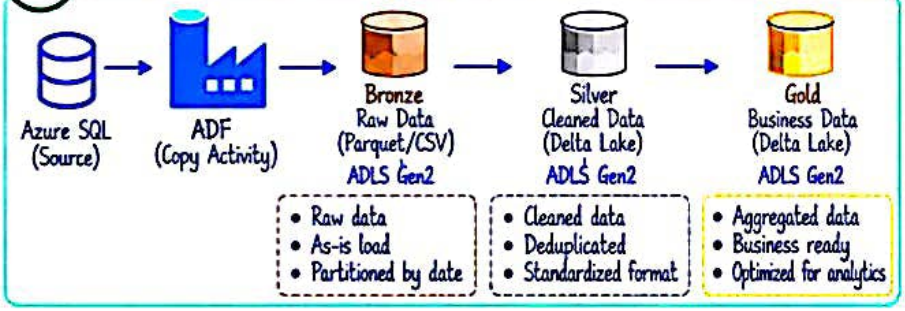
4 ADF Activities Used

- **Copy Activity** — Move data between source and sink
- **Lookup Activity** — Read configuration values
- **Get Metadata** — Get file properties / list of files
- **ForEach Activity** — Process multiple files / tables
- **Databricks Notebook** — Run PySpark transformation
- **If Condition** — Conditional execution
- **Web Activity** — Send notifications (email/Teams)

5 Sample Pipeline Steps



6 Data Flow Between Layers (Medallion Architecture)



7 Best Practices for Production

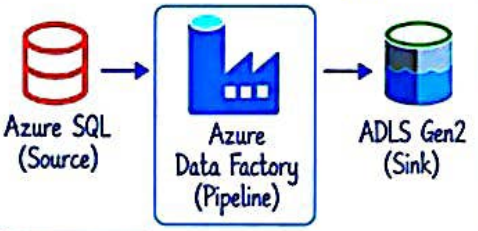
- Use incremental loads (watermark or change tracking).
- Follow medallion architecture (Bronze, Silver, Gold).
- Parameterize pipelines for reusability.
- Use Managed Identity or Key Vault for secure connections.
- Enable retry and failure handling.
- Monitor pipeline runs and set up alerts.
- Use metadata-driven pipelines for multiple tables/files.
- Optimize Databricks notebooks for performance.
- Ensure idempotent design for safe re-runs.
- Track costs and optimize activities and runtimes.

Hands-on & Interview Questions – Azure Data Factory

Build, Monitor and Troubleshoot a Real-World ADF Pipeline

1 Hands-on Challenge

Build an automated pipeline to copy data from Azure SQL to ADLS Gen2 using Azure Data Factory with parameters and schedule trigger.

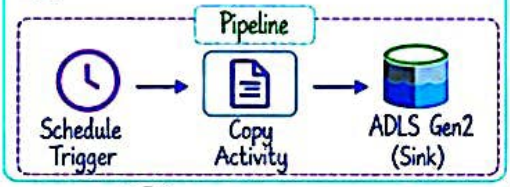


Objectives:

- Ingest data from Azure SQL
- Store in ADLS Gen2 with date-based folder
- Use pipeline parameters
- Schedule daily execution
- Monitor and validate the pipeline

2 Step-by-Step Implementation

1. Create an Azure Data Factory instance in Azure Portal.
2. Create a Linked Service for Azure SQL and ADLS Gen2.
3. Create Datasets for source (Azure SQL) and sink (ADLS Gen2).
4. Add a Copy Activity in the pipeline and configure source and sink.
5. Create pipeline parameters for dynamic file path (e.g. folderPath, fileName).
6. Add a Schedule Trigger (e.g. daily at 2 AM).
7. Run the pipeline and monitor the execution.
8. Test failure handling and retries.



3 Dynamic Content Example

Parameter:

- folderPath : String
- fileName : String
- loadDate : String (optional)

Dynamic File Path Expression:

```
@concat(
    pipeline().parameters.folderPath,
    '/',
    formatDateTime(utcNow(), 'yyyy/MM/dd'),
    '/',
    pipeline().parameters.fileName
)
```

Example Output Path:

sales/2026/10/26/sales.csv

4 Expected Output & Monitoring

ADLS Gen2 Folder Structure



Pipeline Run in Monitor

Activity name	Status	Duration
Copy Sales Data	Succeeded	2m 15s
Lookup Watermark	Succeeded	12s
Get Metadata	Succeeded	8s
ForEach	Succeeded	1m 10s
Databricks Notebook	Succeeded	4m 32s

Sample Data Output

OrderID	CustomerID	OrderDate	Amount
1001	C001	2026-10-26	2500
1002	C002	2026-10-26	1800
1003	C003	2026-10-26	3200

5 Top Interview Questions

1. What is Azure Data Factory?
2. What is the difference between pipeline and activity?
3. What are linked services and datasets?
4. What is Integration Runtime and what are its types?
5. What are the different trigger types in ADF?
6. How do you implement incremental loading in ADF?
7. How do you pass parameters to a pipeline?
8. How do you handle pipeline failures and set up retries?
9. How do you monitor and troubleshoot ADF pipelines?
10. Can we use ADF to call Databricks notebook? How?

6 Best Practices & Key Takeaways

Best Practices

- Use parameterized and reusable pipelines.
- Implement incremental loads using watermark or change tracking.
- Use Managed Identity or Key Vault for secure credentials.
- Enable retry and timeout policies.
- Use meaningful naming conventions.
- Monitor pipeline runs and set up alerts (Azure Monitor / Action Groups).
- Use metadata-driven pipelines for multiple tables/files.
- Test in lower environments before production.
- Follow CI/CD for ADF (ARM templates or Git integration).

Key Takeaways

- ADF automates and orchestrates data integration.
- Supports diverse data sources and compute services.
- Enables scalable and reliable data pipelines.
- Widely used in real-world enterprise data platforms.